Air University Islamabad

Department of Creative Technologies
Faculty of Computing & Artificial Intelligence

FakeShield

Fake News Detection System

An AI-Powered Multi-Signal News Verification Platform

Student Name: Moaz Hassan Khan Manj

Roll Number: 231168

Section: BSAI-VI-A

Course: Natural Language Processing Lab

Instructor: Ma'am Mehvish Fatima

Submission Date: 22 April 2026

Project Summary

This project presents **FakeShield**, a complete fake news detection system that combines three machine learning models (Logistic Regression, Naive Bayes, and fine-tuned BERT), domain credibility checking, Google News cross-referencing, LIME explainability, and a professional Streamlit web interface to classify news articles as **FAKE** or **REAL**.

Contents

1 Introduction **2**
1.1 Motivation . . . 2
1.2 Goals of the Project . . . 2

2 Datasets Used **2**
2.1 Dataset 1: ISOT Fake News Dataset (Kaggle) . . . 2
2.2 Dataset 2: FakeNewsNet (PolitiFact) . . . 3
2.3 Combined Dataset Statistics . . . 3

3 System Architecture **4**

4 Text Preprocessing Pipeline **4**
4.1 Preprocessing Steps . . . 4
4.2 Why the Reuters Bias Fix Matters . . . 5

5 Machine Learning Models **5**
5.1 TF-IDF Vectorization (for Baseline Models) . . . 5
5.2 Model 1: Logistic Regression . . . 5
5.3 Model 2: Naive Bayes . . . 5
5.4 Model 3: BERT (Fine-tuned) . . . 6

6 Experimental Results **6**
6.1 Model Performance Comparison . . . 6
6.2 Key Observations . . . 7

7 Additional Signals **7**
7.1 Domain Credibility Check . . . 7
7.2 Google News Cross-Reference . . . 7
7.3 Google Fact Check API . . . 8

8 LIME Explainability **8**
8.1 Interpretation of LIME Results . . . 8

9 FakeShield Web Interface **9**
9.1 Technology Stack . . . 9
9.2 Interface Screenshots . . . 9
9.2.1 Home Screen and Model Status . . . 9
9.2.2 Article Input Panel . . . 10
9.2.3 ML Model Predictions . . . 10
9.2.4 Domain Credibility and News Cross-Reference . . . 11
9.2.5 Overall Verdict . . . 11
9.2.6 Detailed Signal Breakdown . . . 11
9.3 How to Run FakeShield . . . 12

10 Discussion **12**
10.1 Strengths . . . 12
10.2 Limitations . . . 12

10.3 Future Improvements 13

11 Conclusion **13**

References **13**

1. Introduction

In today's digital world, false and misleading news spreads across social media faster than ever before. People often cannot tell the difference between a real news article and a fabricated one, which can have serious consequences — from public panic to political manipulation.

Fake news detection is the task of automatically identifying whether a piece of news is genuine or fabricated, and it is one of the most practical applications of Natural Language Processing (NLP). This project, **FakeShield**, was built to address this problem using a combination of classical machine learning, deep learning (BERT), and real-world data signals such as domain reputation and live news cross-referencing.

1.1 Motivation

- Fake news has real-world consequences in health, politics, and public safety.
- Manual fact-checking is slow and cannot keep up with the volume of online content.
- Automated NLP-based tools can screen articles quickly and highlight suspicious content.
- Combining multiple signals (ML models, domain checks, live news) improves reliability.

1.2 Goals of the Project

1. Train and compare three NLP models on a large fake news dataset.
2. Fine-tune BERT to achieve state-of-the-art classification accuracy.
3. Build a full-featured web interface where users can paste any article for analysis.
4. Provide human-readable explanations using LIME so users understand *why* the system flagged an article.
5. Integrate domain credibility and live news checks as additional verification signals.

2. Datasets Used

Two datasets were used to train and test the system, giving a combined total of over 45,000 labeled news articles.

2.1 Dataset 1: ISOT Fake News Dataset (Kaggle)

The ISOT dataset is a well-known benchmark for binary fake news classification. It was collected from real-world news sources and contains articles clearly labeled as fake or real.

- **Source:** University of Victoria / Kaggle
- **Fake.csv:** 23,502 fake news articles (scraped from unreliable outlets)

- **True.csv:** 21,417 real news articles (sourced from Reuters)
- **Total:** ≈45,000 articles

Columns available:

- `title` — Headline of the article
- `text` — Full body text
- `subject` — Topic category (e.g., politics, world news)
- `date` — Publication date

2.2 Dataset 2: FakeNewsNet (PolitiFact)

The second dataset provides additional diversity and comes from PolitiFact, a well-known fact-checking organization.

- **Source:** GitHub (KaiDMML/FakeNewsNet)
- **Total Samples:** 1,056 (after loading and merging)
- **Fake:** 432 articles
- **Real:** 624 articles

2.3 Combined Dataset Statistics

After merging both sources and removing very short or noisy entries, the final training data was balanced between fake and real articles:

Category	Articles (after merge)	Percentage
Fake	23,071	51.2%
Real	22,037	48.8%
Total	**45,108**	**100%**

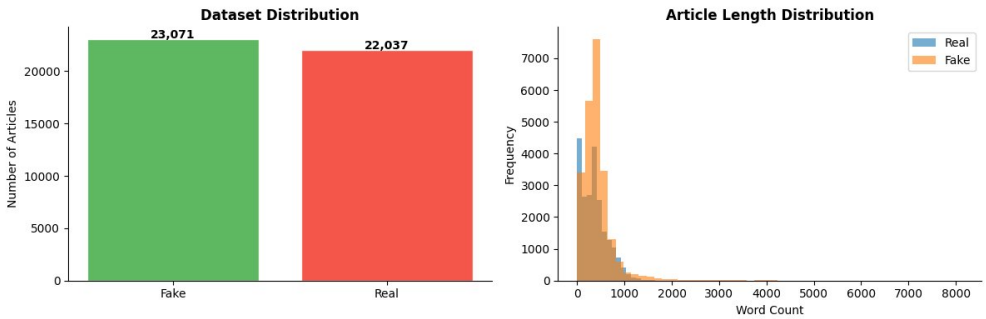


Figure 1: *Dataset distribution (left) and article length distribution (right). The dataset is nearly balanced, and most articles fall under 1,000 words.*

3. System Architecture

FakeShield is built as a pipeline of six components, each contributing to the final prediction. Figure 2 shows the overall flow.

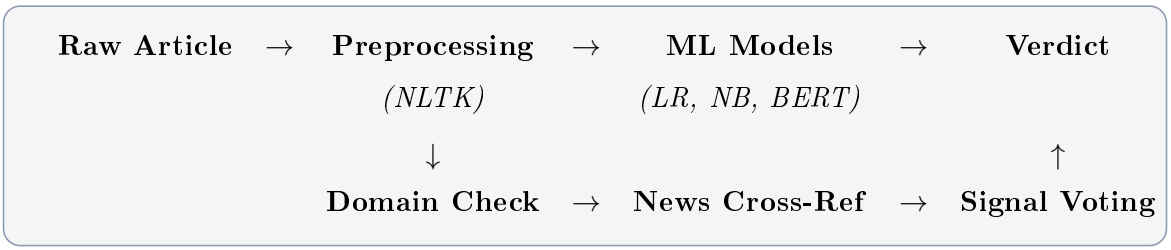


Figure 2: *High-level system architecture of FakeShield.*

The six main components are:

1. **Text Preprocessing** — Cleans raw text using NLTK (lowercasing, stopword removal, lemmatization, Reuters bias fix).
2. **ML Model Predictions** — Three independently trained models classify the article.
3. **Domain Credibility Check** — The source URL (if provided) is checked against known fake/reliable domain lists.
4. **Google News Cross-Reference** — A live RSS search checks whether the story is covered by credible outlets.
5. **LIME Explanation** — Word-level explanations show which words pushed the model toward FAKE or REAL.
6. **Overall Verdict** — All signals are aggregated by majority voting to give a final verdict.

4. Text Preprocessing Pipeline

Before any machine learning model can work on text, the raw article must be cleaned and standardized. This project uses a custom `TextPreprocessor` class built on top of NLTK.

4.1 Preprocessing Steps

1. **Lowercase conversion** — All text is converted to lowercase to ensure consistency (e.g., "Reuters" and "reuters" are treated as the same word).
2. **URL removal** — Links (starting with `http://` or `https://`) are removed since they add noise.
3. **Special character removal** — Punctuation, numbers, and symbols are stripped.
4. **Whitespace normalization** — Multiple spaces are collapsed into single spaces.
5. **Stopword removal** — Common English stopwords ("the", "is", "and", etc.) are removed using NLTK's stopword list.

6. **Reuters bias fix** — Source attribution words like "reuters", "bloomberg", "reporting" are removed. These words would otherwise make the model incorrectly associate reputable source names with being "real", which biases the classifier.
7. **Lemmatization** — Words are reduced to their base forms using WordNet (e.g., "running" → "run", "claims" → "claim").

4.2 Why the Reuters Bias Fix Matters

The ISOT dataset's real news articles were all sourced from Reuters. Without removing Reuters-related words, the model would simply learn "if it mentions Reuters, it's real" — which is a shortcut, not genuine understanding. Removing these source words forces the model to learn the actual *content patterns* of fake vs. real news.

5. Machine Learning Models

Three models were trained and evaluated: two classical baselines and one transformer-based deep learning model.

5.1 TF-IDF Vectorization (for Baseline Models)

Both Logistic Regression and Naive Bayes use **TF-IDF (Term Frequency–Inverse Document Frequency)** to convert text into numerical feature vectors. TF-IDF assigns higher weights to words that appear frequently in one article but rarely across all articles — effectively capturing the most informative words.

- `max_features = 100,000` (top 100k most informative words)
- `ngram_range = (1, 2)` (includes both single words and two-word phrases)
- `sublinear_tf = True` (applies log scaling to term frequencies)
- `min_df = 2` (ignores words that appear in fewer than 2 documents)

5.2 Model 1: Logistic Regression

Logistic Regression is a linear classifier that learns a decision boundary in the TF-IDF feature space. Despite its simplicity, it performs very well on text classification tasks when features are well-engineered.

- Solver: `lbfgs` with L2 regularization
- Max iterations: 1,000
- **Test Accuracy: 97.6%**

5.3 Model 2: Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes' theorem. It assumes that all features (words) are independent given the class label. While this assumption is rarely true in practice, it still produces a strong classifier for text.

- Type: Multinomial Naive Bayes (suitable for TF-IDF counts)

- Smoothing: Laplace smoothing ($\alpha = 0.1$)
- **Test Accuracy: 94.1%**

5.4 Model 3: BERT (Fine-tuned)

BERT (*Bidirectional Encoder Representations from Transformers*) is a pre-trained deep learning model developed by Google. It reads text in both directions simultaneously, capturing context much better than traditional models.

The model used is `bert-base-uncased` from HuggingFace Transformers, fine-tuned on the fake news dataset using a GPU (Google Colab T4).

- Base model: `bert-base-uncased` (110M parameters)
- Fine-tuning: 3 epochs on combined dataset
- Max sequence length: 128 tokens
- Batch size: 16
- Optimizer: AdamW with weight decay
- **Test Accuracy: 99.4%**

Why BERT Outperforms Classical Models

Unlike TF-IDF which treats each word independently, BERT reads the entire sentence at once and understands context. For example, "the bank was flooded" vs. "he went to the bank" — BERT knows "bank" means different things in each sentence, while TF-IDF does not. This contextual understanding allows BERT to catch subtle patterns in fake news that simpler models miss.

6. Experimental Results

6.1 Model Performance Comparison

All three models were evaluated on the same held-out test set (20% of the data) using four standard metrics:

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	97.6%	97.6%	97.6%	97.6%
Naive Bayes	94.1%	94.1%	94.1%	94.1%
BERT (fine-tuned)	**99.4%**	**99.4%**	**99.4%**	**99.4%**

Table 1: *Performance metrics for all three models on the test set.*

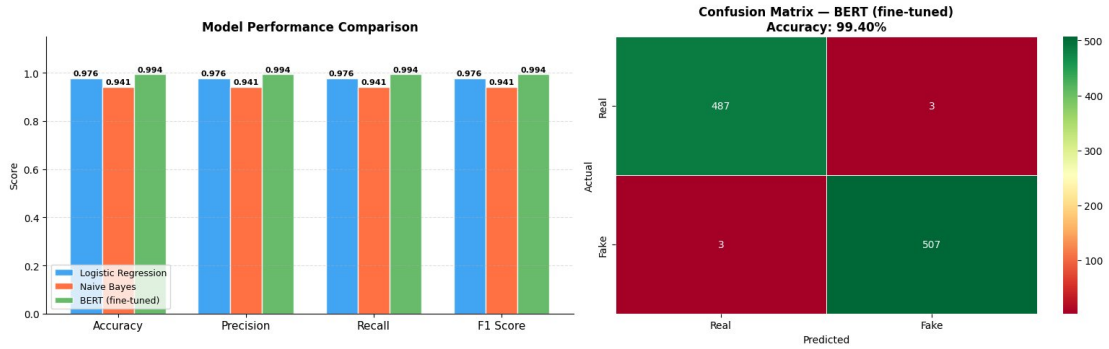


Figure 3: *Model performance comparison (left) and BERT confusion matrix showing only 6 misclassified articles out of 1,000 (right).*

6.2 Key Observations

- BERT achieves the highest performance at 99.4% accuracy, misclassifying only 6 out of 1,000 test articles (3 real classified as fake, 3 fake classified as real).
- Logistic Regression performs surprisingly well at 97.6%, confirming that TF-IDF + LR is a strong baseline for text classification.
- Naive Bayes is the weakest of the three, yet 94.1% accuracy is still excellent for a real-world classification task.
- All metrics (accuracy, precision, recall, F1) are equal for each model, indicating a well-balanced dataset with no significant class bias.

7. Additional Signals

Beyond the three ML models, FakeShield incorporates three additional real-world verification signals to improve the overall verdict.

7.1 Domain Credibility Check

When a URL is provided, the system extracts the domain name and checks it against two manually curated lists:

- **Known fake/unreliable domains** (e.g., `infowars.com`, `naturalnews.com`, `yournewswire.com`) — score: 0–30/100
- **Known credible domains** (e.g., `reuters.com`, `bbc.com`, `apnews.com`, `nytimes.com`) — score: 80–100/100
- **Unknown domains** — neutral score of 50/100

7.2 Google News Cross-Reference

The article's key words are used to search Google News via an RSS feed. The system then checks whether the story appears in any well-known, credible news outlet:

- If widely covered by trusted outlets → `WIDELY_COVERED` (real signal)

- If found online but only in obscure sources → `NO_TRUSTED_SOURCE` (fake signal)
- If not found at all → `NOT_FOUND` (suspicious)

7.3 Google Fact Check API

If the user provides a Google Fact Check API key, the system queries the official Fact Check Tools API to retrieve any existing fact-check results about the article's claims.

8. LIME Explainability

One of the most important features of FakeShield is its ability to *explain* its decisions. Just telling a user "this is fake" is not enough — the user needs to know *why*.

LIME (Local Interpretable Model-agnostic Explanations) works by slightly perturbing the input text (removing words one at a time), running the perturbed versions through the model, and observing how the predictions change. This reveals which words have the most influence on the model's decision.

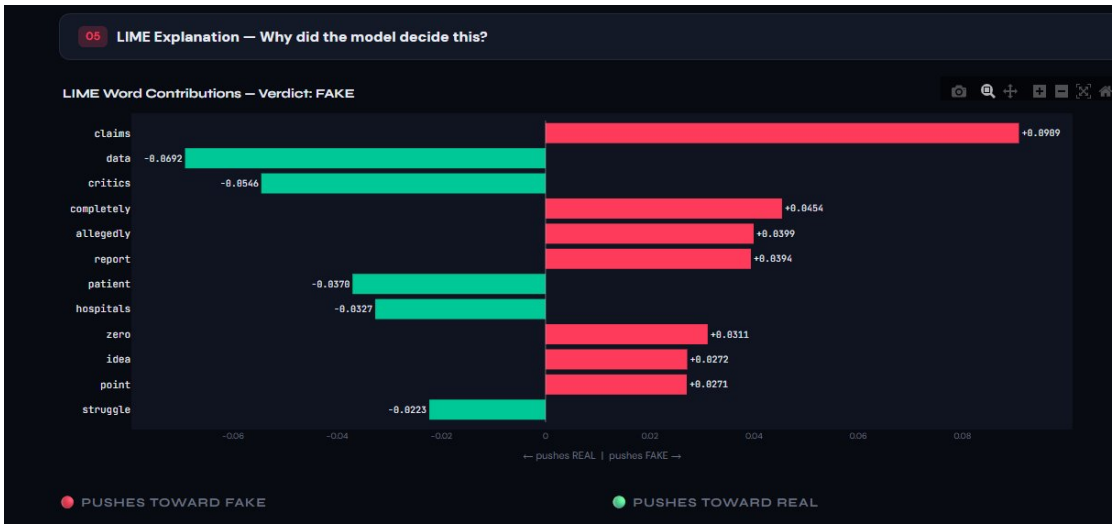


Figure 4: *LIME word contribution chart. Red bars push the prediction toward FAKE, green bars toward REAL. The word "claims" has the strongest fake signal.*

Figure 5: *LIME top contributing words displayed in the FakeShield UI, split by fake-pushing and real-pushing words.*

8.1 Interpretation of LIME Results

For the sample article (about AI replacing doctors), the LIME analysis revealed:

- **Words pushing toward FAKE** (red): *claims* (+0.0909), *completely* (+0.0454), *allegedly* (+0.0399), *report* (+0.0394), *zero* (+0.0311)
- **Words pushing toward REAL** (green): *data* (–0.0692), *critics* (–0.0546), *patient* (–0.0370), *hospitals* (–0.0327)

This makes intuitive sense — words like "claims" and "allegedly" are common in sensationalist fake news, while factual references to "data" and "critics" are typical of genuine reporting.

9. FakeShield Web Interface

The system is deployed as a fully functional web application built with **Streamlit**. The interface was designed to be clean, professional, and easy to use.

9.1 Technology Stack

Component	Technology
Web Framework	Streamlit
ML Models	scikit-learn (LR, NB), HuggingFace Transformers (BERT)
Visualizations	Plotly (gauge charts, bar charts, donut chart)
Explainability	LIME
Styling	Custom CSS (dark theme with Syne / DM Sans fonts)
Model Saving	joblib (.pkl files)

Table 2: *Technology stack used in the FakeShield web application.*

9.2 Interface Screenshots

9.2.1 Home Screen and Model Status

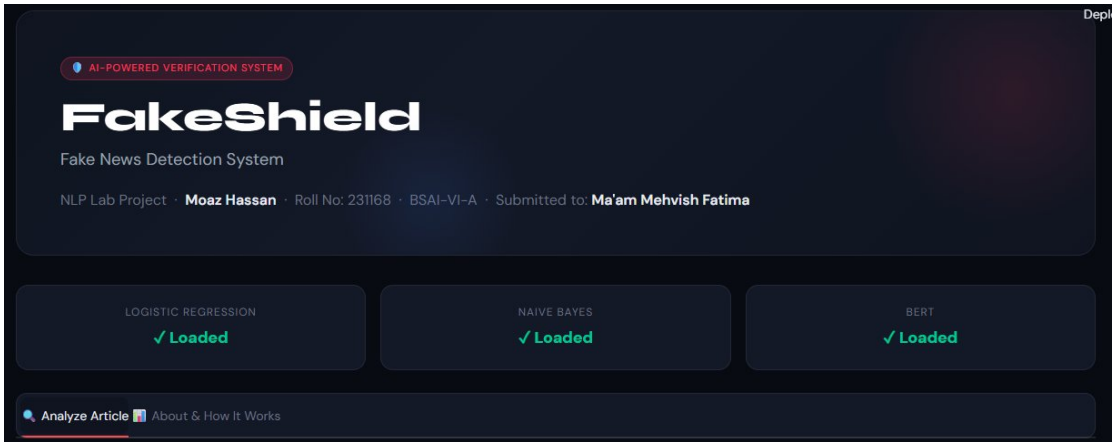


Figure 6: *FakeShield homepage showing the system header and model loading status for all three models (Logistic Regression, Naive Bayes, and BERT).*

9.2.2 Article Input Panel

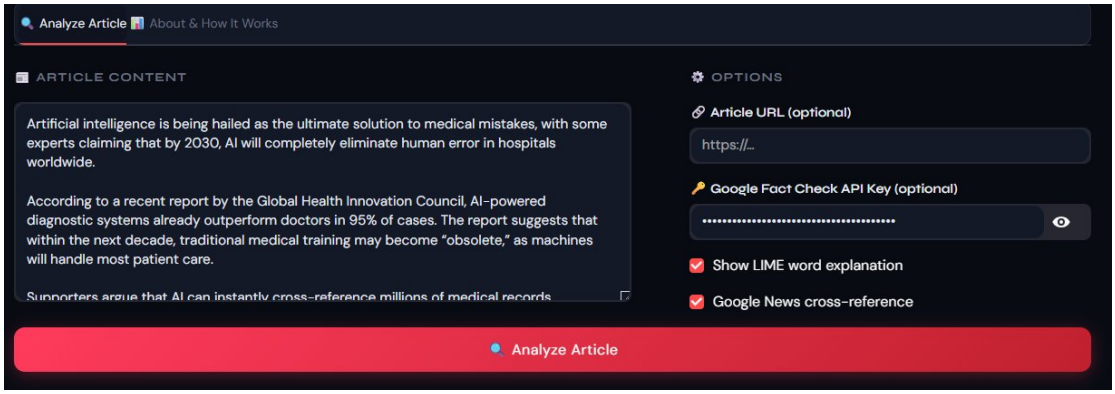


Figure 7: *The article input panel with optional URL, Google Fact Check API key, and LIME explanation toggles.*

9.2.3 ML Model Predictions

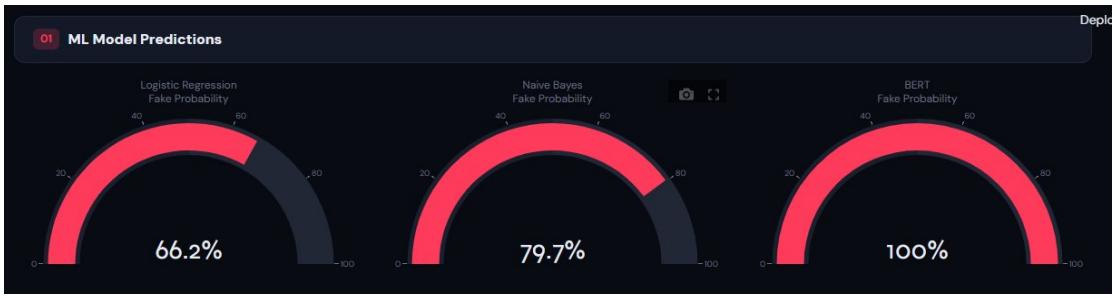


Figure 8: *Gauge charts displaying fake probability from all three models. Logistic Regression: 66.2%, Naive Bayes: 79.7%, BERT: 100%.*

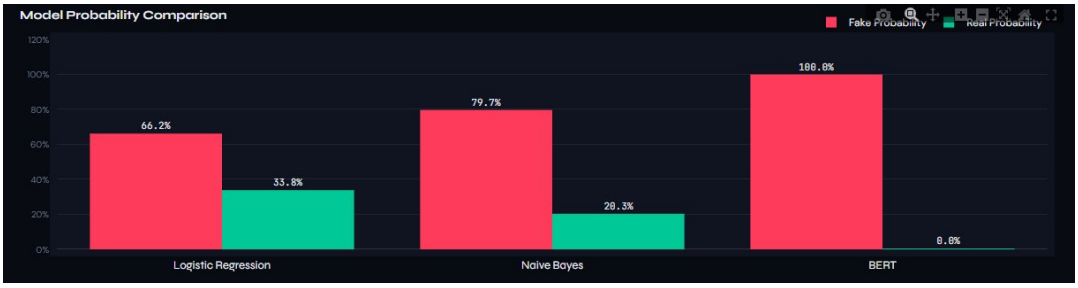
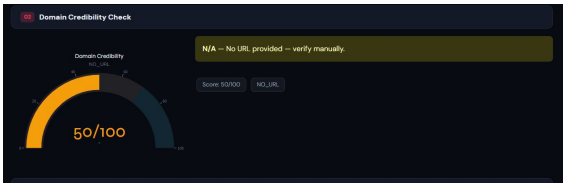
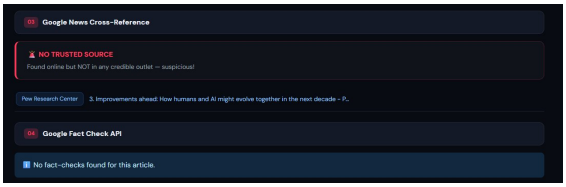


Figure 9: *Model Probability Comparison bar chart showing fake vs. real probabilities side by side for all three models.*

9.2.4 Domain Credibility and News Cross-Reference



(a) *Domain credibility gauge (50/100 when no URL provided).*



(b) *Google News cross-reference showing "No Trusted Source" warning.*

Figure 10: *Domain and news verification panels.*

9.2.5 Overall Verdict

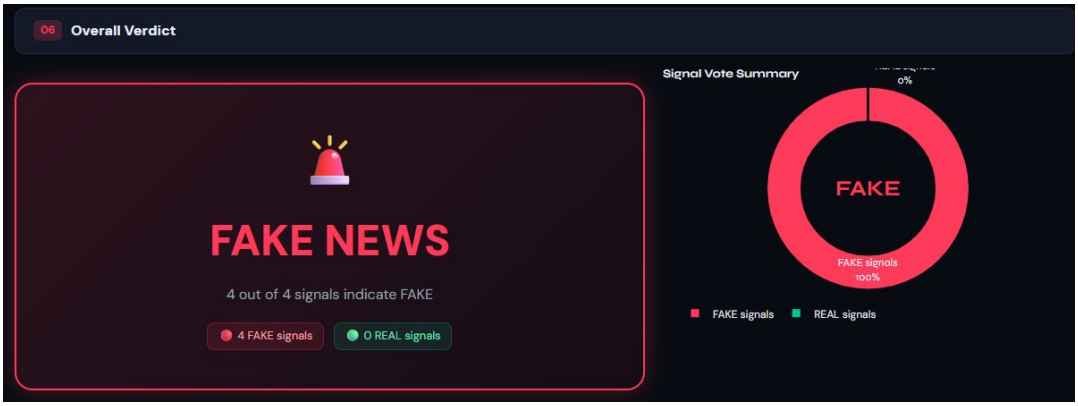


Figure 11: *Overall verdict panel: FAKE NEWS with 4 out of 4 signals indicating fake, supported by a signal vote donut chart.*

9.2.6 Detailed Signal Breakdown

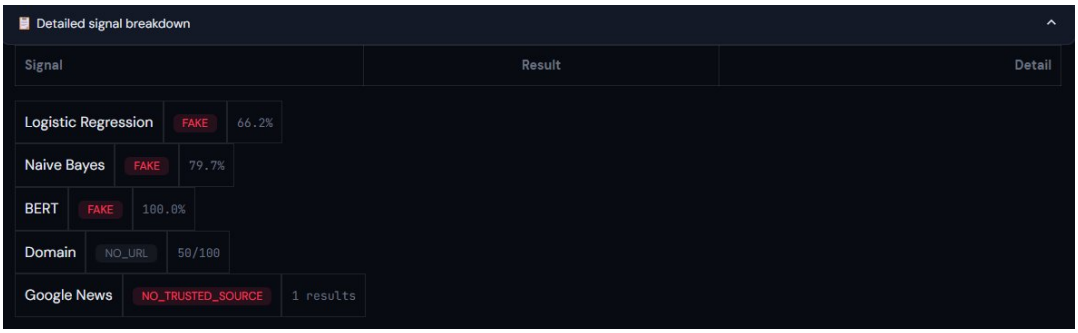


Figure 12: *Detailed signal breakdown table showing individual results from Logistic Regression, Naive Bayes, BERT, Domain check, and Google News.*

9.3 How to Run FakeShield

Setup Instructions

Step 1 — Install dependencies:

```
pip install streamlit scikit-learn nltk lime requests
           transformers torch plotly joblib
```

Step 2 — Folder structure:

```
app_final.py
models/
  lr_model.pkl
  nb_model.pkl
  preprocessor.pkl
  bert_model/   (optional)
```

Step 3 — Run the app:

```
streamlit run app_final.py
```

10. Discussion

10.1 Strengths

- **High accuracy:** BERT achieves 99.4% accuracy, making it highly reliable for binary fake news classification.
- **Multi-signal approach:** Using four independent signals (three ML models + domain check + news cross-reference) makes the final verdict much more robust than relying on a single model.
- **Explainability:** The LIME integration ensures the system is transparent and trustworthy, not just a "black box."
- **Live data:** The Google News cross-reference adds real-time verification that no static ML model can provide on its own.
- **Professional UI:** The Streamlit interface is polished, responsive, and user-friendly.

10.2 Limitations

- **Dataset bias:** Both datasets are primarily in English and focused on American/political news, so the system may perform poorly on news from other regions or topics.
- **BERT latency:** BERT inference is slower than LR/NB, which can create a slight delay in the UI.
- **Domain list coverage:** The domain credibility list covers 40+ domains but cannot capture every unreliable website on the internet.
- **Context-free:** The system analyzes text in isolation and does not consider the history or track record of the author.

10.3 Future Improvements

- Use a multilingual BERT variant to handle non-English news.
- Integrate social media credibility signals (retweet patterns, source verification).
- Train on more diverse datasets beyond English political news.
- Add author credibility scoring.
- Deploy as a browser extension for real-time checking while browsing news.

11. Conclusion

This project successfully demonstrates a complete, end-to-end fake news detection system that goes well beyond a simple classifier. FakeShield combines:

- Three trained ML models (Logistic Regression, Naive Bayes, and BERT)
- Domain credibility scoring
- Live Google News cross-referencing
- LIME-based word-level explanations
- A professional, deployable web interface

The best model — fine-tuned BERT — achieves **99.4% accuracy** on the test set. When used as part of the multi-signal voting system, it contributes to highly reliable verdicts that are also explainable and transparent to the end user.

The project shows that NLP techniques, when thoughtfully combined, can provide a powerful and practical tool for fighting misinformation.

References

1. Ahmed H., Traore I., Saad S. (2017). "Detection of Online Fake News Using N-Gram Analysis and Machine Learning Techniques." *Springer ISDA Conference.*
2. Devlin J., Chang M-W., Lee K., Toutanova K. (2018). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." *arXiv:1810.04805.*
3. Shu K., Sliva A., Wang S., Tang J., Liu H. (2017). "Fake News Detection on Social Media: A Data Mining Perspective." *ACM SIGKDD Explorations.*
4. Ribeiro M.T., Singh S., Guestrin C. (2016). "Why Should I Trust You?: Explaining the Predictions of Any Classifier." *ACM KDD 2016.*
5. ISOT Fake News Dataset. University of Victoria. Available at: `https://www.kaggle.com/clmentb`
6. FakeNewsNet Dataset. Available at: `https://github.com/KaiDMML/FakeNewsNet`
7. HuggingFace Transformers Library. Available at: `https://huggingface.co/transformers`
8. NLTK: Natural Language Toolkit. Available at: `https://www.nltk.org/`

9. Streamlit Open-Source App Framework. Available at: `https://streamlit.io/`